

Fog of War without Zero-Knowledge Proofs: A Blinded Mutual-Sighting Test for Trustless Game Channels

Andrew Colosimo Roy Crombleholme Andrew Gore
Konstantin Gorskov Daniel Kraft

Xaya Developers

July 27, 2026

Abstract

Replicated deterministic state machines, blockchains and the state channels built on them, make hidden information awkward: every replica must be able to verify every transition, yet fog of war requires that positions stay secret. Commit and reveal hides positions but destroys *encounters*: two players walk past each other in the dark because neither may learn that the other is near. We present a *blinded mutual-sighting test*. After both players have hash-committed their moves for a round, and before either move is disclosed, the two clients run a Diffie–Hellman-blinded private membership test, the Meadows 1986 and Huberman–Franklin–Hogg 1999 lineage cut down from set intersection to a single membership bit evaluated in both directions. Both parties learn exactly one shared bit, “we are in sight”, and nothing else. Two design choices adapt the classical protocol to a consensus setting. First, every blinding scalar and pad element is derived deterministically from a seed the player is already bound to reveal, which makes the entire honest transcript recomputable by an after-the-fact audit. Second, no conviction ever trusts the exchange itself: cheating is adjudicated by recomputing the public sight predicate over positions fixed by commitments made before the round’s information existed. Together these replace an in-round zero-knowledge proof, which does not fit the execution budget of our referee, with after-the-fact attribution, at essentially no consensus cost on the honest path.

1 Introduction

Multiplayer games on public replicated state machines inherit a contradiction. The chain, or the channel, exists so that every participant can verify every transition; fog of war exists so that participants *cannot* see part of the state. The standard resolution, commit now and reveal later, solves only half of the problem. A hash commitment hides a player’s position perfectly, and perfect hiding is precisely what makes the game unplayable: if neither player may learn anything about the other’s position before the endgame, then two players a few tiles apart walk past each other in the dark, every round, for the whole match. In our own testbed, removing all position disclosure produced long stretches of mutual line of sight in which neither player was ever told anything, which is an invisible non-game.

The requirement is therefore a conjunction:

1. **Privacy.** Nobody learns anything about your position unless they are close to you, or unless you spend it deliberately, for instance on a contested objective.

2. **Encounters.** Two players who *are* close must both learn so, in the same round, every round, with no exceptions.
3. **Adjudication.** Every rule above must remain enforceable by a referee that never sees secret state, on a metered execution budget.

The natural cryptographic answer to requirement 3 is a zero-knowledge proof attached to every move. That answer is unavailable to us in-round: our referee is a zero-import WebAssembly blob with a hard fuel budget per call, and a single Groth16 verification costs more than the whole budget. This paper describes what we built instead.

Contributions.

1. A *blinded mutual-sighting test* (§5): a two-party private membership test, run in both directions between move commitment and move disclosure, that yields the one shared bit $\text{sight}(a, b)$ and nothing else. The construction is the classical Diffie–Hellman private-matching protocol of Meadows and of Huberman, Franklin and Hogg [1, 2], specialised from $\text{set} \cap \text{set}$ to membership of a single element.
2. A *determinism-for-auditability* discipline (§7): every byte a party should send is a pure function of a seed it is already committed to reveal, so a judge can recompute any disputed transcript after the fact. This converts “prove your input was honest”, which is a proof system’s job, into “lying is non-repudiable evidence”, at negligible consensus cost.
3. *Enforcement by geometry, not cryptography* (§7): no conviction ever depends on the exchange. Suppressing a sighting, and disputing a phantom one, are both adjudicated by recomputing the public sight predicate over positions bound by hash commitments made before the round’s information existed.

2 Setting and threat model

Game channels. The system runs on *game channels* [18] as deployed on the Xaya platform: two players play a turn-based game off-chain, exchanging signed moves, with the blockchain as the fallback arbiter. A single host process referees many different games by loading each game’s rules as a hash-pinned, zero-import WebAssembly blob; the blob is fuel-metered per call and its state is size-capped. Both clients and every potential arbiter run the identical blob, so “the judge” below means *any* party replaying the rules; honesty of the judge is by replication, not by assumption.

Round structure. Play proceeds in rounds on a fixed public map, in our deployment a tile dungeon generated procedurally from a shared seed. Each round:

1. **Commit.** Both players publish hash commitments to their move, position and action, salted by a per-match secret seed_i .
2. **Sighting test.** The exchange of §5, between the two clients only. The referee is not involved.
3. **Disclose.** If the test said *no*, a player publishes nothing about its position. If it said *yes*, both players are obliged to publish their exact positions for the round, with an arrival proof, and on first contact the connecting path.

Every match ends with an **endgame audit**: both players reveal seed_i and their full paths, which the blob verifies against every round’s commitments and replays against the map. This audit predates the sighting test; §7 extends it.

Adversary. A malicious client controls all of its own messages: it may send arbitrary bytes in the sighting exchange, stall, or disclose falsely. It cannot break hash commitments or signatures, and it cannot avoid the endgame audit except by forfeiting, since timeouts convert stalling into forfeit. Both players’ clients may be malicious simultaneously; the game between two colluding cheaters is not our concern, but neither cheater may gain against an honest opponent. The design rule throughout: *consensus never trusts a claim*, and everything that convicts is recomputed by the blob from committed data.

3 The sight predicate

Positions are walkable tiles of the public map. For tiles a, b let $d(a, b)$ be the shortest walkable path distance and let $\text{los}(a, b)$ be the game’s line-of-sight predicate, a symmetric visibility test on the tile grid. For a fixed sight range ρ the predicate jointly computed each round is

$$\text{sight}(a, b) := [d(a, b) \leq \rho] \wedge \text{los}(a, b). \quad (1)$$

Here d is symmetric by construction, and los was verified symmetric exhaustively over all in-range walkable pairs of the test maps. Symmetry makes “the” shared bit well defined: writing

$$V(x) := \{t \text{ walkable} : \text{sight}(x, t)\}, \quad a \in V(b) \iff b \in V(a), \quad (2)$$

with $x \in V(x)$ always, since distance 0 satisfies (1). The protocol pads every set to a fixed size N chosen above the largest $|V|$ the map admits (§4.1), so message size and work are constants independent of the sender’s surroundings.

The predicate must be symmetric for the construction to apply at all: asymmetric sight ranges would make the two parties’ bits legitimately disagree and the shared-bit framing collapse. Consequently the game defines exactly one sight range in consensus, and light sources are cosmetic.

4 Cryptographic requirements

The protocol is not tied to a particular group. It needs three properties, and only the third is easy to get wrong.

1. **Commuting blinding.** A group operation and scalars for which $\alpha(\beta X) = \beta(\alpha X)$. Any prime-order group serves, and so does RSA-style multiplicative blinding; nothing below depends on the choice.
2. **Hardness.** Distinguishing $\{\alpha P_t : t \in T_0\}$ from $\{\alpha P_t : t \in T_1\}$ for candidate sets T_0, T_1 , given the public P_t , must be hard. In a prime-order elliptic-curve group this is a standard decisional Diffie–Hellman problem.
3. **Elements with unknown discrete logarithms.** The map $t \mapsto P_t$ from a game position to a group element must be a hash-to-group map whose outputs have discrete logarithms unknown to everyone, including the party doing the hashing. This is load-bearing rather than stylistic, for the reason given in §6.

Our instantiation. We use Curve25519 [14] with x-only Montgomery arithmetic, the RFC 7748 [16] ladder, and an x-only Elligator 2 [15] map with the cofactor cleared at construction so that every point lies in the prime-order subgroup. The map is a single-map encoding rather than the RFC 9380 [17] suite: the protocol needs determinism, unknown discrete logarithms and prime-order outputs; it does not need the output to be indistinguishable from uniform, because every transmitted element is additionally randomised by a per-round blinding scalar. Interoperability is a non-goal, since every party runs the same pinned blob. The choice matters only for cost, and the reference implementation [13] carries the details, the known-answer vectors, and the argument for each deviation.

4.1 Derivations: nothing is random

Every quantity in the exchange is derived rather than sampled. This is the part that makes the audit of §7 possible, so it is a protocol requirement and not an implementation detail:

$$\text{position elements (public): } P_t = \text{H2C}(\text{"T"} \parallel \text{enc}(t)), \quad (3)$$

$$\text{blinding scalar of player } i, \text{ round } r: \alpha_r^{(i)} = \text{SHA-256}(\text{"P"} \parallel \text{seed}_i \parallel \text{enc}(r)), \quad (4)$$

$$\text{pad dummies of player } i: D_j^{(i)} = \text{H2C}(\text{"D"} \parallel \text{seed}_i \parallel \text{enc}(j)), \quad j = |V|, \dots, N-1, \quad (5)$$

with distinct domain-separation tags, and with α reduced into the scalar range and the measure-zero zero case patched deterministically. The dummies of (5) are intentionally round-independent: per-round freshness comes entirely from α_r , and fixed base elements let a client precompute every group element it will ever hash, which is the expensive half of the per-round cost.

Crucially, seed_i is the same secret that already salts player i 's move commitments and that is revealed in full at the endgame audit. That reuse is the design: it makes the honest transcript a pure function of data the player is already bound to expose.

5 The protocol

Fix round r ; both players have committed position and action for r . Write $\alpha_A = \alpha_r^{(A)}$, and let a, b be the committed positions. Before any disclosure:

Flight 1 (both directions).

$$A \rightarrow B: \quad S_A = \text{sort}\left(\left\{\alpha_A P_t : t \in V(a)\right\} \cup \left\{\alpha_A D_j^{(A)} : j = |V(a)|, \dots, N-1\right\}\right), \quad Q_A = \alpha_A P_a, \quad (6)$$

and symmetrically $B \rightarrow A$ sends (S_B, Q_B) . Here S is N elements plus the single element Q , and sort orders by encoding, a canonical order independent of enumeration that reveals nothing about which entries are dummies. Exactly $N+1$ hash-to-group maps and $N+1$ scalar multiplications are performed whatever $|V|$ is, so neither length nor timing leaks the sender's surroundings.

Flight 2 (both directions).

$$A \rightarrow B: \quad R_A = \alpha_A Q_B (= \alpha_A \alpha_B P_b), \quad B \rightarrow A: \quad R_B = \alpha_B Q_A (= \alpha_B \alpha_A P_a). \quad (7)$$

Local finish.

$$\text{bit}_A = [R_B \in \{\alpha_A s : s \in S_B\}], \quad \text{bit}_B = [R_A \in \{\alpha_B s : s \in S_A\}], \quad (8)$$

each costing N scalar multiplications and N comparisons.

Theorem 1 (Correctness). *Except with probability negligible in the hash length, $\text{bit}_A = \text{bit}_B = \text{sight}(a, b)$.*

Proof sketch. On the prime-order group, $\alpha_A(\alpha_B P_t) = \alpha_B(\alpha_A P_t)$, so $R_B = \alpha_A \alpha_B P_a$ equals $\alpha_A(\alpha_B P_t)$ if and only if $P_a = P_t$, that is if and only if $a \in V(b)$, unless two distinct hash-to-group inputs collide, which is bounded by the birthday term over the domain-separated inputs. Thus $\text{bit}_A = [a \in V(b)]$ and symmetrically $\text{bit}_B = [b \in V(a)]$, and these coincide with $\text{sight}(a, b)$ by (2). With x-only encodings P and $-P$ are identified, but hash-to-group outputs are themselves x-coordinates, so that coincidence is the same collision event and is already counted. $\square$

The deployed client asserts that the two bits agree on every round for which it can observe both, and treats disagreement as a defect rather than as an input.

Reference implementation. An open-source implementation of the protocol, together with an independent reimplementaion written against the protocol’s specification in a separate codebase with no shared code, is available [13]. The two reproduce each other’s flights, responses and finishing bits byte for byte, so the specification and the deployed system cannot silently diverge.

6 Security analysis

The designed leakage is the bit, and, by padding, nothing about $|V|$.

The blinded set hides its contents. Recovering anything about $V(a)$ from S_A is the distinguishing problem of §4, requirement 2. The receiver holds nothing else involving α_A .

Unknown discrete logarithms are load-bearing. The blinding must be by a secret factor and the elements being blinded must have unknown discrete logarithms. If positions were instead encoded as $P_t = h_t G$ with h_t a public hash of t , then every element’s discrete logarithm would be public, and a receiver of one flight could strip the blinding: each h_v is public and invertible, so multiplying a received element by h_v^{-1} for candidate positions v yields αG at the correct guess, after which one further pass labels every element of the set. Over a game’s small universe of positions this is a few thousand scalar multiplications, so the sender’s set, and hence its position, is recovered from a single honest protocol run. With a hash-to-group map the corresponding first step, obtaining αG from αP_t without knowing $\log_G P_t$, is the computational Diffie–Hellman problem. This is also why brute force over a small universe buys nothing: “is this element position t ?” has no checkable relation without breaking Diffie–Hellman.

Remark 1. This is not a hypothetical. A published reference implementation of ECDH-PSI constructs its elements exactly this way [10]; we reported it upstream, and it was fixed. It is the sharpest illustration we know of why the hash-to-group map is not a stylistic detail.

Cross-round unlinkability. α_r is fresh per round, so linking $\alpha_r P_t$ to $\alpha_{r+1} P_t$, a stationary player, is again a Diffie–Hellman distinguishing problem. The dummies being fixed per seed changes nothing, since they are reblinded like every position.

Reflection and replay. B may echo one of A ’s own set elements back as its Q_B , giving $R_A = \alpha_A^2 P_t$. Exploiting the pair $(\alpha_A P_t, \alpha_A^2 P_t)$ is the square Diffie–Hellman problem, for which no attack is known in the groups we use. The bit that B computes from a garbage Q is garbage, which is self-sabotage, and it becomes attributable evidence under §7.

Malformed elements. A malicious peer can substitute an arbitrary string for a group element. In our x-only instantiation the ladder processes off-curve and low-order inputs without special-casing; the worst case is a small-order element sent as Q , whose reply αQ reveals α modulo the cofactor, a handful of bits out of the scalar’s full width, and useless against the distinguishing problem. The reply never matches a cofactor-cleared set element, so the resulting bit is 0. Like every malformed flight this is self-sabotage now and evidence later.

The accumulated-negative leak. Every “no” round tells a player that the opponent is outside its visible set, and these constraints accumulate. This is the designed privacy and liveness trade of the whole scheme rather than an oversight: any protocol that guarantees requirement 2 of §1 must leak at least the negatives, because a guaranteed encounter is exactly the absence of a false negative. We measured the residual anonymity set on our own maps against an exact Bayesian attacker granted every advantage, and it remains large enough to play with, but the figure is a property of a particular map, sight range and movement speed rather than of the protocol, and it must be re-derived for any other setting.

What the exchange deliberately does not provide. First, no in-round proof that V was computed from the committed position: a malicious party can feed any set or Q it likes, and the surrounding system makes this pointless or attributable rather than impossible (§7). Second, no fairness or liveness: a party can stall mid-exchange, and the channel’s existing timeout forfeits cover it. Third, it requires the predicate to be symmetric (§3).

7 Attribution instead of zero-knowledge

The referee never verifies the exchange in-round, so the honest path costs consensus nothing. Three facts combine into the enforcement story.

1. **The honest transcript is a pure function.** By §4.1, every byte party i should send in round r is $T(\text{seed}_i, r, \text{position}_i(r), \text{map})$: the scalar from the seed, V from the committed position and the public map, position elements and dummies from fixed derivations, and the order fixed by the sort.
2. **The audit reveals the preimages.** The pre-existing endgame audit forces both players to reveal seed_i and their full paths, verified against every round’s hash commitments. Any judge can therefore recompute, after the fact, the exact bytes an honest party would have sent in any round, and compare them with what was sent. Each party publishes a commitment to the flight it sent inside its own signed move, which makes a wrong byte non-repudiable. The recomputation must be of a *single named element* rather than of a whole flight: rebuilding a flight inside the referee is orders of magnitude beyond its per-call budget, whereas verifying a supplied element against the committed flight and recomputing that one element is cheap. Using the commitment the sender already publishes, rather than a signature scheme inside the blob, is what keeps this affordable.

3. **Convictions never trust the bit.** “In sight and stayed dark” and “disputed a sighting that was not there” are both adjudicated by recomputing **sight**, Eq. (1), over positions bound by commitments made before the round’s information existed. The bit only tells honest clients when they are obliged to act; the map convicts.

So a proof system’s job, *prove your input was honest*, is replaced by *your input is a deterministic function of things you are already bound to reveal, and lying about it is signed evidence*. The guarantee is delivered after the fact instead of in-round, which a turn-based channel game with a mandatory endgame audit can afford, and the consensus-side cost is essentially zero.

The trust boundary, stated plainly. Attribution closes a gap that is open in both directions. A malicious opponent who fabricates its flight can make an honest client’s instrument read a phantom “yes”, causing it to over-disclose or to file a dispute; and it can equally make the instrument read a false “no”, blinding the honest client so that it fails an obligation it was never told it had. The second is the more dangerous, because it is a winning strategy rather than a nuisance: blind an opponent every round, watch it approach unaware, then escalate and dispute it, and a referee that judged the contradiction on geometry alone would convict the blinded victim.

Both close only if the commitment to the flight PRECEDES the exchange, which is why it rides the commitment move rather than the disclosure: the commit pass closes before any set is published, so every participant’s root is public before any flight exists on the wire. The response needs no commitment of its own, since the audit recomputes it from the revealed seed and the peer’s committed element; publishing it is what makes a doctored response attributable. What the adjudication then establishes, on the two positions the audit has verified, is which side’s committed bytes contradict the geometry: whether the disputer’s own set contained the element that would have informed its victim, and whether the accused’s set produced the hit the disputer reports. Convictions therefore follow from committed data and never from either party’s claimed bit, which is precisely the quantity an attacker forges. A false accusation is not free, and neither is a blinding carried out through the committed set.

One further property is required, and it is worth stating precisely because it is a property of the transport rather than of the protocol: the bytes a party DELIVERS must be the bytes it is later judged against. An earlier version of our implementation exchanged the flights off-chain and bound the response only by publication in a disclosure — a disclosure ordered, by the dispute’s own filing rules, after the victim had already had to choose whether to reveal. A party could therefore deliver a doctored response, or a doctored set, off-chain while publishing the honest values, and the recipient could not distinguish doctored from honest without the sender’s secret scalar. What the audit can recompute is what SHOULD have been sent, never what WAS; delivery is unprovable without attribution on the delivered bytes, so no adjudication rule over off-chain messages closes it. The deployed protocol therefore delivers nothing off-chain at all. Both legs of the exchange are PUBLISHED as signed channel messages in a pass of their own, after every commitment is fixed and before any disclosure: the referee refuses a published set that does not rebuild the committed root, each response is computed from the peer’s committed element read out of the published state — so the exchange needs no delivered message anywhere — and delivery and publication coincide by construction, at which point the recomputation convicts a doctored response exactly as it convicts a doctored set.

8 Related work

The Diffie–Hellman private-matching lineage. The double-blinding trick at the core of §5 is old and good. Meadows [1] used commutative blinding for cryptographic matchmaking in 1986, and Huberman, Franklin and Hogg [2] used the same structure for private community matching in 1999. The modern literature knows it as ECDH-PSI: formalised and extended by Freedman, Nissim and Pinkas [5] and given linear-complexity instantiations by De Cristofaro and Tsudik [6]. It runs at planetary scale today, with Google’s Password Checkup [8] and Apple’s PSI system [9] both descendants. Our protocol is this classical construction *reduced*: from $\text{set} \cap \text{set}$ to membership of a single element, evaluated in both directions, because the game needs one shared bit and not the overlap.

Private set intersection for fog of war. The closest prior art to this work is OpenConflict [11]. Motivated by a practical memory-snooping map hack against real-time strategy games, it distributes game state so that each client holds only what it is entitled to see, using a private set intersection over grid cells, and pads the sets so that unit counts do not leak. It reports 22 ms per synchronisation at the peak of a professional StarCraft II game, so the approach is fast enough for real time and not merely for turn-based play. The padding idea in §4.1 is theirs. Two things differ here. OpenConflict assumes that players cannot lie about their inputs, having neither a referee nor a commitment layer, whereas our setting is a consensus engine in which a client may submit anything at all and the entire question is what happens afterwards; and our predicate is a joint sighting test adjudicated by a replicated referee, rather than a state-distribution mechanism among clients who are trusted to be passive.

Applying that line of work to a blockchain game, with positions hash-committed on chain and revealed at the end of the match for deterministic replay, was sketched by Thomson [12], who also stated plainly that the case of actively dishonest players remained unsolved. That is the gap §7 addresses: replaying positions and actions does not detect a fabricated *protocol* input, because nothing in such a scheme binds the exchanged messages to the positions later revealed. Deriving every blinding scalar from a committed seed is what closes it.

The implementation we started from. Our engineering entry point into this literature was Thomson’s open-source ECDH-PSI implementation [10], a clear reference implementation of the classical protocol with an encrypted-payload layer for revealing which elements matched. Studying and prototyping against it shaped what we kept and what we changed: we adopted an explicit hash-to-group map, for the reason in §6; dropped the payload-encryption layer entirely, since we must never learn *which* element matched and so have nothing to decrypt; fixed the set cardinality by padding, since otherwise $|V|$ leaks for free; and, most importantly, bound the inputs. Like most PSI reference implementations that one leaves inputs unbound, so a malicious party can use the protocol as a membership oracle over fabricated sets, and our committed-seed determinism and audit (§7) exist precisely to close that gap.

Private equality and proximity testing. Comparing secrets without revealing them goes back at least to Fagin, Naor and Winkler [3]. The closest prior art to our setting is Narayanan et al.’s private proximity testing [7], which reduces location proximity to private equality of grid cells for social location-sharing. Our test differs in the predicate, walkable-path distance plus line of sight rather than Euclidean grid overlap; in being mandatory and bidirectional every round, since their users choose when to query whereas our players are *obliged* to act on the bit and the

obligation is enforced by consensus; and in the adversary handled, since their protocols assume honest-but-curious phones whereas our clients are fully malicious and handled by attribution.

Hidden information in adversarial games. Mental Poker [4] founded the field of playing hidden-information games without a trusted dealer. The modern blockchain line is exemplified by Dark Forest [19], which achieves fog of war on a public chain by attaching a zk-SNARK to every move, proving it valid relative to hidden coordinates. That is the “prove your input was honest” path we could not take, since our referee’s per-call budget is below the cost of a single Groth16 verification, and channel games settle on chains where verifier costs bind harder still. The determinism-plus-audit construction of §7 is our substitute, and it is available to any turn-based game that already ends in a full reveal. Note also the difference in what is proved: Dark Forest proves unilateral facts about one player’s hidden move, whereas a sighting is an irreducibly *joint* predicate over two players’ secrets, which is what forces a two-party protocol rather than a proof.

Infrastructure. Game channels are due to Kraft [18], deployed on Xaya, the successor of Huntercoin, to our knowledge the first blockchain to run a multiplayer game world fully on-chain, and whose scaling and privacy limits motivated this line of work. Curve25519 is Bernstein’s [14], the Elligator 2 map is from Bernstein, Hamburg, Krasnova and Lange [15], the ladder and its test vectors follow RFC 7748 [16], and RFC 9380 [17] is the general hash-to-curve framework.

9 Limitations and future work

1. **Attribution is deterrence, not prevention.** A participant who fabricates a flight learns the answer it wanted before the audit convicts it, so the stake must exceed the value of the information. At more than two participants a dispute is also terminal for the whole table rather than for the pair: the audit it forces reveals every participant’s seed, and revealed seeds cannot continue a hidden-information game, so the match settles on its merits — the same property the demand-an-immediate-end move has by design.
2. **A one-round rotation edge.** A suppressor whose disclosure happens to be ordered after the honest player’s can dodge a dispute for a single round of fleeting edge-of-range contact.
3. **Privacy measured pairwise.** The deployed implementation runs the exchange at up to four participants — each round every participant publishes one response per peer, and the dispute adjudication names its pair explicitly — at per-round cost linear in the number of pairs. The anonymity-set measurements of §6, however, are two-participant; more observers can only shrink the hider’s candidate set, and we have not yet quantified by how much.
4. **Cost.** The per-round curve arithmetic is a fraction of a turn-based beat and roughly halves with precomputation of the fixed elements of (3) and (5), which is why those derivations are round-independent. Much shorter beats would need batching or a faster field implementation.
5. **The leak is a dial, not zero.** The accumulated-negative constraint is inherent to guaranteed encounters, and we chose to measure the resulting anonymity set rather than claim more than the construction gives.

Acknowledgments

Edward Thomson’s open-source PSI implementation [10] was the practical starting point of this work: a readable, working embodiment of the classical protocol that let us prototype in days and decide what a consensus setting had to change. The protocol itself stems from four decades of prior art beginning with Meadows [1]; our contribution is the adaptation, not the trick.

References

- [1] C. Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In *IEEE Symposium on Security and Privacy*, 1986.
- [2] B. A. Huberman, M. Franklin, and T. Hogg. Enhancing privacy and trust in electronic communities. In *ACM Conference on Electronic Commerce (EC)*, 1999.
- [3] R. Fagin, M. Naor, and P. Winkler. Comparing information without leaking it. *Communications of the ACM*, 39(5), 1996.
- [4] A. Shamir, R. L. Rivest, and L. M. Adleman. Mental poker. Technical Report MIT-LCS-TM-125, Massachusetts Institute of Technology, 1979.
- [5] M. J. Freedman, K. Nissim, and B. Pinkas. Efficient private matching and set intersection. In *EUROCRYPT*, 2004.
- [6] E. De Cristofaro and G. Tsudik. Practical private set intersection protocols with linear complexity. In *Financial Cryptography and Data Security*, 2010.
- [7] A. Narayanan, N. Thiagarajan, M. Lakhani, M. Hamburg, and D. Boneh. Location privacy via private proximity testing. In *Network and Distributed System Security Symposium (NDSS)*, 2011.
- [8] K. Thomas et al. Protecting accounts from credential stuffing with password breach alerting. In *USENIX Security Symposium*, 2019.
- [9] A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe. The Apple PSI system. Apple Inc. technical report, 2021.
- [10] E. A. Thomson. Private-Set-Intersection: an ECDH-based PSI implementation. <https://github.com/EdwardAThompson/Private-Set-Intersection>, accessed July 2026.
- [11] E. Bursztein, M. Hamburg, J. Lagarenne, and D. Boneh. OpenConflict: preventing real time map hacks in online games. In *IEEE Symposium on Security and Privacy*, 2011.
- [12] E. A. Thomson. Preventing cheaters in fog of war games. <https://edward-thomson.medium.com/preventing-cheaters-in-fog-of-war-games-69f202fbe107>, June 2020.
- [13] Xaya Developers. fog-of-war: a blinded sighting test, reference implementation. <https://github.com/xaya/fog-of-war>, 2026.
- [14] D. J. Bernstein. Curve25519: new Diffie–Hellman speed records. In *Public Key Cryptography (PKC)*, 2006.

- [15] D. J. Bernstein, M. Hamburg, A. Krasnova, and T. Lange. Elligator: elliptic-curve points indistinguishable from uniform random strings. In *ACM Conference on Computer and Communications Security (CCS)*, 2013.
- [16] A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, 2016.
- [17] A. Faz-Hernández, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood. Hashing to elliptic curves. RFC 9380, 2023.
- [18] D. Kraft. Game channels for trustless off-chain interactions in decentralized virtual worlds. *Ledger*, 1:84–98, 2016. <https://doi.org/10.5195/ledger.2016.15>.
- [19] Dark Forest team. Dark Forest: a zkSNARK-based incomplete-information game on Ethereum. <https://zkga.me>, 2020.